



RAG with AWS OpenSearch: Cloud Vector DB

A deep dive into production-scale retrieval-augmented generation — from local development to cloud-ready search infrastructure.

Before We Start

RAG Basics

- Do I understand why retrieval matters more than model memory for factual answers?
- Do I know how chunks, embeddings, and context injection work together?
- Do I understand that this session is about adapting RAG for production-scale search?
- If these ideas still feel mixed together, that is normal — the comparisons will separate them.

Production Thinking

- Do I know what makes a retrieval system production-ready?
- Can I think about scale, reliability, versioning, and change management?
- Do I understand that a good cloud choice is not only about features, but also operations?
- Gaps here are fine; this session is about the trade-offs.



How do ChromaDB and OpenSearch compare?

Local vs. Managed ChromaDB is a local vector store; OpenSearch is a managed cloud search service with a fundamentally different operational model.	ChromaDB Strengths A strong choice for fast local development and simple setup — minimal configuration, zero infrastructure overhead.
OpenSearch Strengths A better fit when scale, availability, and shared access matter — built for production workloads.	Operational Cost OpenSearch brings extra operational responsibility: cluster management, networking, and monitoring.

 **The main idea:** ChromaDB favors simplicity, while OpenSearch favors production readiness.

Why would a team move from ChromaDB to OpenSearch?

→ **Scale**

To handle larger document sets and more traffic without hitting local resource limits.

→ **Reduced Infrastructure Burden**

To reduce local infrastructure responsibility and offload operational overhead to a managed service.

→ **Shared Retrieval**

To support shared or centralized retrieval across multiple services, teams, or applications.

→ **Production Patterns**

To move closer to production deployment patterns and align with enterprise architecture standards.

→ **Operational Resilience**

To make retrieval more operationally resilient with built-in redundancy and failover.

What makes a retrieval system production-ready?



Reliable Indexing

Reliable indexing and repeatable retrieval — every query returns consistent, trustworthy results.



Scalability

Ability to handle growth in documents and traffic without degrading performance.



Observability

Clear observability and debugging paths so issues can be identified and resolved quickly.



Safe Updates

Safe handling of document updates and model changes without breaking existing retrieval.



Stability Under Change

Stability under change, not just good search accuracy — the system must hold up as requirements evolve.



Why does backend choice matter?

Performance & Operations

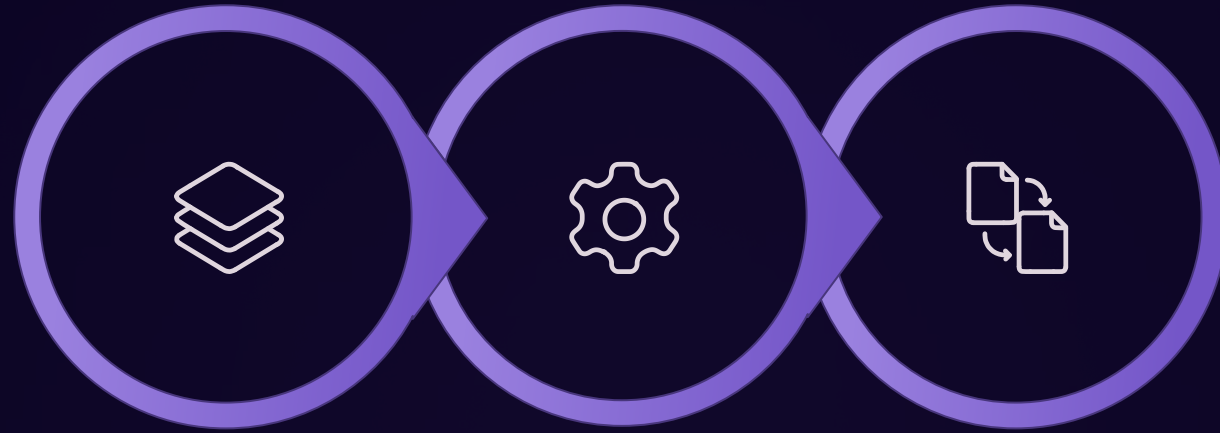
- It affects latency and scale — the backend determines how fast and how far your system can go.
- It changes setup, maintenance, and deployment work for your engineering team.
- It influences how easy the system is to operate day-to-day.

Architecture Impact

- It changes the trade-off between simplicity and resilience at every layer.
- The backend is part of the architecture, not just a detail — it shapes everything above it.

❏ Choose your backend as deliberately as you choose your model.

How should retrieval backends be switched?



**Separate
Logic**

Configure

**Preserve
Behavior**

A clean separation of concerns means the agent never needs to know which backend is running — only the configuration layer changes.

Separate Logic

Keep retrieval logic separate from agent logic.

Use Configuration

Use configuration to choose ChromaDB or OpenSearch.

Agnostic Interface

Keep a backend-agnostic retrieval interface.

Preserve Behavior

Preserve the same agent behavior when the backend changes.

What stays constant when the backend changes?

Agent Prompt & Response Flow

The agent's prompting strategy and how it processes responses remain completely unchanged.

Retrieval Interface

The contract between the agent and the retrieval layer stays the same regardless of what's underneath.

Grounding & Fallback Behavior

How the agent grounds answers and handles missing context is unaffected by backend swaps.

User-Facing Behavior

End users should see no difference — the experience is identical regardless of backend.

The Only Thing That Changes

Only the **storage and search layer** should change. Everything above it — the agent, the interface, the user experience — remains stable and consistent.

□ This is the power of a well-abstracted retrieval architecture.



Why is chunking important in RAG?



Retrieval Precision

Chunking helps retrieval stay precise — smaller, focused units are easier to match against a query.



Context Preservation

It helps preserve enough context for the answer — chunks must carry enough meaning to be useful.



Retrieval Scope

Chunking determines what the retriever can find — the granularity of chunks defines the search space.



Answer Quality

It directly affects answer quality — poor chunking leads to poor answers, regardless of model quality.

 Chunking is a balance between **context** and **specificity** — neither extreme serves the user well.

What chunk sizes are commonly used?



Small: 200–400 tokens

Best for focused facts and precise retrieval where specificity matters most.



Medium: 400–800 tokens

Balanced context — the most common choice for general-purpose RAG applications.



Large: 800+ tokens

When broader context matters — useful for narrative documents or complex reasoning tasks.

The best size depends on the document type and use case. Overly large chunks can dilute relevance; overly small chunks can lose meaning.

What are widely used chunking methods?

1

Fixed-Size Chunking

Split documents into equal-length segments by token or character count. Simple and predictable.

2

Overlapping Chunking

Adjacent chunks share a window of tokens to prevent meaning from being cut at boundaries.

3

Semantic Chunking

Split at natural semantic boundaries — sentences, topics, or meaning shifts detected by a model.

4

Structure-Aware Chunking

Use document structure — headings, sections, paragraphs — to define chunk boundaries.

5

Hybrid Chunking

Combine several methods to get the benefits of each — often the best approach for complex documents.

How should PDFs be chunked?

Recommended Approach

Chunk by headings and paragraphs

Use document structure when possible rather than arbitrary page breaks.

Use overlap

Page layout can split meaning — overlap ensures context is not lost at boundaries.

Preserve nearby sections

When text spans multiple pages, keep related sections together in the same chunk.

Key Considerations

PDFs often need **structure recovery** before chunking — raw PDF extraction can be noisy and disordered.

A policy PDF is usually better chunked **by section** than by page — pages are a layout artifact, not a semantic unit.

⚠ Never assume PDF page boundaries align with meaning boundaries.



How should DOCX or document files be chunked?



Chunk by sections, headings, and subheadings

Use the document's own hierarchy as the primary chunking signal.



Keep related paragraphs together

Paragraphs under the same heading usually share context and should stay in the same chunk.



Use structure-aware chunking when formatting is clean

Well-formatted DOCX files reward structure-aware approaches with high-quality chunks.



Preserve section order and hierarchy

Maintain the document's logical flow — order and nesting carry meaning.

✓ DOCX files usually give **better structure** than PDFs — take advantage of it.

How should CSV files be chunked?

Chunking Strategy

Row-by-row for simple records

When each row is self-contained, chunk individually for maximum retrieval precision.

Group related rows

When one row alone is not enough context, group logically related rows together.

Include column headers

Always include column headers with every chunk — without them, values lose meaning.

Preserving Meaning

Preserve table meaning

Chunk for semantic completeness, not just raw cells — a cell without context is useless.

Row context plus metadata

CSVs often work best when row content is paired with metadata like source, date, or category.

How should tables be handled?

Keep Header Context

Keep header context with each chunk — column labels are essential for interpreting any cell value.

Avoid Fragmentation

Avoid breaking tables into meaningless fragments — a partial row or orphaned column is worse than no data.

Store as Metadata

Store row identity and column labels as metadata alongside the chunk for richer retrieval filtering.

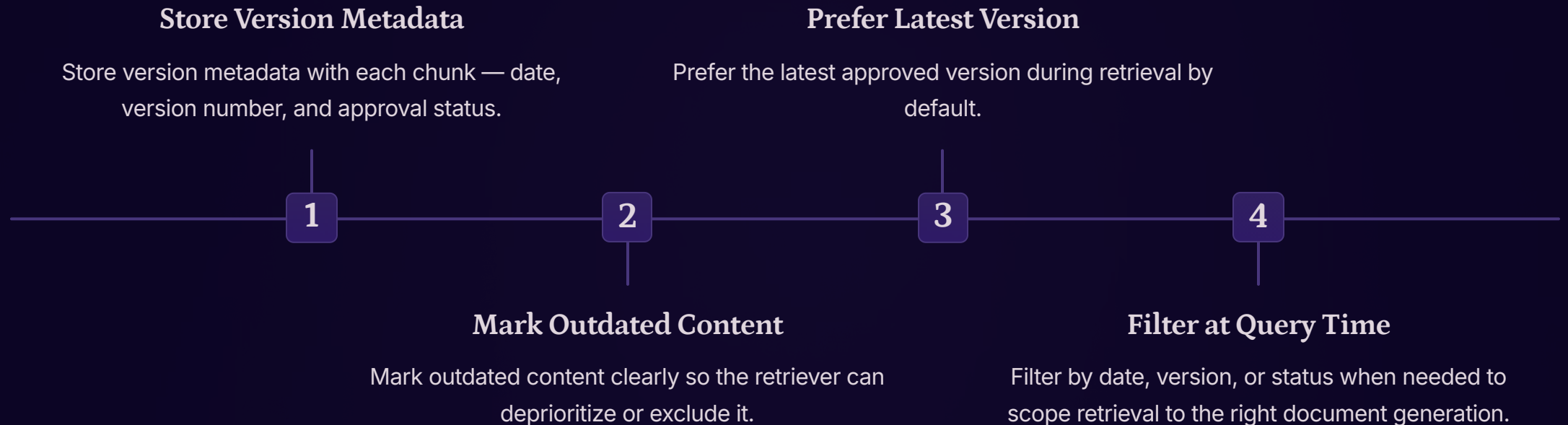
Table-Aware Extraction

Use table-aware extraction tools when possible — generic text extractors often destroy table structure.

 Tables need context to remain interpretable — never treat them as plain text.



How do you handle multiple versions of the same document?



✓ **Version control is about trust as much as freshness** — users need to know the answer comes from the right source.

What can go wrong without version handling?

The Risk

Without version metadata, your retrieval system has no way to distinguish between current and outdated content — and neither does your user.

- ⊗ Version metadata reduces this risk — but only if it is consistently applied at ingestion time.

Failure Modes

- **Old policies retrieved first**

Outdated documents can rank higher than current ones if version signals are absent.

- **Conflicting answers in one session**

Multiple versions of the same document can produce contradictory answers in a single conversation.

- **Outdated product or policy information**

Users may receive information that was accurate months ago but is no longer valid.

- **Correct answer, wrong version**

The agent may answer correctly based on an old version — technically accurate but practically wrong.

Why do embedding model changes matter?

Different Vector Spaces

Different embedding models create different vector spaces — the same text produces entirely different numerical representations.

Compatibility Requirement

Query and document embeddings must stay compatible — mixing models from different versions breaks similarity search.

Search Quality Impact

Search quality can change after a model swap — sometimes dramatically, in either direction.

Full Reindex Required

A new embedding model may require a full reindex of all documents — a significant operational event.

⚠ **Embedding changes are a retrieval schema change**, not a cosmetic update — treat them with the same rigor as a database migration.



What should happen when the embedding model changes?



Re-embed the document set

Run all documents through the new embedding model to generate fresh vector representations.



Rebuild or refresh the index

Rebuild the OpenSearch index with the new embeddings — do not mix old and new vectors in the same index.



Keep indices separate during migration

Keep old and new indices separate during migration if needed — this allows safe rollback if issues arise.



Validate retrieval quality

Validate retrieval quality before switching fully — run benchmark queries against both indices.



Test correctness and recall

Test both correctness and recall after the change — a model may improve precision but reduce recall, or vice versa.